МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»

Кафедра телекоммуникационных систем и вычислительных средств (ТС и ВС)

РАСЧЕТНО-ГРАФИЧЕСКАЯ РАБОТА
по дисциплине
«WEB—технологии»

по теме:
СОЗДАНИЕ ТЕСТОВОЙ ВИДЕОПЛАТФОРМЫ С ИСПОЛЬЗОВАНИЕМ ТЕХНОЛОГИЙ PYTHON ДЛЯ БЭКЕНДА, REACT JS ДЛЯ ФРОНТЕНДА И FIGMA ДЛЯ ДИЗАЙНА

Студент:
Группа № ИА-232 *И.А. Зырянов*

Преподаватель:
Старший предподаватель *А.В. Андреев*

Новосибирск 2026 г.

СОДЕРЖАНИЕ

1 ВВЕДЕНИЕ 3

2 ХОД РАБОТЫ 5

2.1 Выбор технологий и общая архитектура 5

2.2 Структура проекта и организация бэкенда 5

2.3 Модель данных и реализация API 6

2.4 Фронтенд и взаимодействие с API 7

2.5 Пользовательский интерфейс 7

2.6 Трудности и их решение 7

2.7 Запуск проекта 8

2.8 Интерфейс и соответствие макету Figma 8

3 ВЫВОДЫ 11

1 ВВЕДЕНИЕ

Современные веб-приложения, предназначенные для работы с медиаконтентом, как правило, реализуются по клиент-серверной архитектуре. Пользователь взаимодействует с интерфейсом через браузер, который отправляет запросы к серверной части приложения (API). Обработка данных происходит на стороне сервера, а сами данные и файлы хранятся в базе данных и файловой системе либо в специализированных хранилищах. Для реализации учебной видеоплатформы используется следующий стек технологий: фронтенд на React, бэкенд на Python, реляционная база данных PostgreSQL, а также Docker для упрощения развёртывания среды.

Цель расчётно-графической работы — разработка тестовой видеоплатформы, предоставляющей пользователю возможность просматривать список видеороликов, загружать собственные видео, изменять информацию о них и удалять записи. Также предусмотрена регистрация пользователя и последующее использование учётной записи при работе с приложением. Интерфейс должен быть удобным для пользователя, соответствовать базовым принципам UI/UX и корректно отображаться на устройствах с различной шириной экрана.

Задачи работы:

- разработать и реализовать REST API на языке Python (с использованием фреймворка FastAPI), обеспечивающее хранение данных пользователей и информации о видео в базе PostgreSQL, а также загрузку и отдачу видеофайлов;
- реализовать механизм регистрации пользователя и хранение состояния авторизации на стороне клиента (с использованием React и `localStorage`);
- создать клиентскую часть приложения на React с использованием маршрутизации (React Router), включающую каталог видео, страницу регистрации и страницу просмотра видео с комментариями;
- реализовать пользовательский интерфейс в соответствии с макетом, разработанным в Figma, с учётом цветовой схемы, расположения элементов и адаптивности;

– обеспечить удобную структуру проекта, подготовить инструкции по запуску и описать процесс разработки, включая возникшие трудности и способы их решения.

Практическая значимость работы заключается в закреплении навыков разработки современных веб-приложений, включая проектирование API, работу с базами данных, интеграцию клиентской и серверной частей, а также решение типичных проблем, возникающих при разработке (настройка CORS, работа с портами, взаимодействие с Docker и согласование адресов между фронтендом и бэкендом).

2 ХОД РАБОТЫ

2.1 Выбор технологий и общая архитектура

В рамках задания допускалось использование Django или Flask, однако для реализации серверной части был выбран **FastAPI**. Данный фреймворк позволяет быстро создавать REST API, поддерживает асинхронную обработку запросов и включает встроенную валидацию данных с помощью Pydantic. По своей сути это также веб-приложение на Python, обрабатывающее HTTP-запросы и маршруты.

Фронтенд: в качестве клиентской части использована библиотека **React 18** совместно со сборщиком **Vite**. Для навигации между страницами применён **React Router v6**. Информация о текущем пользователе сохраняется в `localStorage` и используется через контекст (`AuthContext`), что позволяет сохранять состояние после обновления страницы.

Хранение данных: используется **PostgreSQL 16**. Подключение реализовано через библиотеку **psycopg** без применения ORM. Создание таблиц выполняется при запуске приложения с помощью SQL-запросов, что упрощает развертывание проекта.

Работа с видео: загружаемые файлы сохраняются в директорию `backend/uploads` с уникальным именем. Для доступа к ним клиенту передаётся относительный путь, который затем используется во фронтенде для формирования полного URL.

2.2 Структура проекта и организация бэкенда

Проект разделён на две основные части: `frontend/` и `backend/`. В серверной части выделены следующие модули:

- `config.py` — содержит настройки приложения (параметры базы данных, CORS, путь к загрузкам);
- `db.py` — отвечает за подключение к базе данных и создание таблиц;
- `schemas.py` — описывает структуры данных для запросов и ответов;

– `serializers.py` — преобразует данные из базы в формат, удобный для отправки клиенту;
– `routers/` — содержит обработчики маршрутов (пользователи, видео, комментарии, проверка состояния сервиса).

Такое разделение облегчает поддержку проекта и делает код более понятным.

2.3 Модель данных и реализация API

В базе данных создаются таблицы: `users`, `videos`, `video_comments` и `comment_likes`. В них хранится информация о пользователях, видеороликах, комментариях и отметках «нравится».

Основные API-методы:

– `POST /register` — регистрация пользователя;
– `POST /access-code/request` — получение кода доступа;
– `GET, POST, PUT, DELETE /videos` — операции с видеозаписями (просмотр, добавление, редактирование, удаление);
– методы для работы с комментариями и лайками.

Для проверки работы сервера используется эндпоинт `/health`, возвращающий ответ `{"status": "ok"}`.

Пример инициализации приложения:

```
def create_app() -> FastAPI:
    application = FastAPI(title="Video Platform API")
    application.mount("/uploads", StaticFiles(directory=UPLOAD_DIR), 
    application.add_middleware(CORSMiddleware, allow_origins=FRONTEND
    application.include_router(health_router)
    application.include_router(users_router)
    application.include_router(videos_router)
    return application
```

2.4 Фронтенд и взаимодействие с API

Клиентская часть содержит несколько основных страниц: каталог видео, регистрация и просмотр конкретного видео. Для переходов между ними используется React Router.

Запросы к серверу вынесены в отдельный модуль, где задаётся базовый адрес API. Это позволяет централизованно обрабатывать ответы и ошибки.

Воспроизведение видео осуществляется с помощью HTML-тега `<video>`, который получает ссылку на файл, расположенный на сервере. Файл загружается по HTTP-запросу.

2.5 Пользовательский интерфейс

Интерфейс разработан на основе макета из Figma. Были использованы карточки для отображения видео, шапка сайта и кнопки действий. Для адаптации под разные устройства применены медиазапросы.

Главная страница содержит каталог видео, страница регистрации — форму ввода данных пользователя, а страница просмотра — видеоплеер и блок комментариев.

2.6 Трудности и их решение

В процессе разработки возникли следующие проблемы:

Настройка Docker. Иногда возникали ошибки при загрузке образов. Решением стало повторное выполнение команды или использование локальной установки базы данных.

Проблемы с CORS. Из-за разных портов фронтенда и бэкенда браузер блокировал запросы. Это было исправлено добавлением соответствующих настроек в FastAPI.

Работа с путями к файлам. Требовалось корректно формировать URL для доступа к видео. Для этого был использован единый базовый адрес API.

Работа интерфейса для незарегистрированного пользователя. Были добавлены подсказки и переход на страницу регистрации.

2.7 Запуск проекта

Для запуска базы данных используется Docker:

```
cd backend/docker
docker compose up -d
```

Запуск серверной части:

```
cd backend
uvicorn app.main:app --reload
```

Запуск клиентской части:

```
cd frontend
npm install
npm run dev
```

2.8 Интерфейс и соответствие макету Figma

Пользовательский интерфейс разработан на основе макета, созданного в Figma. В оформлении используются фоновое изображение, контрастная верхняя панель, карточки видео с визуальным выделением, а также единый стиль заголовков и кнопок. Для корректного отображения на различных устройствах применяются медиазапросы, позволяющие изменять количество колонок в каталоге.

На рисунке 1 представлен внешний вид главной страницы для незарегистрированного пользователя. В данном случае отображается пустой каталог и подсказка о необходимости регистрации.

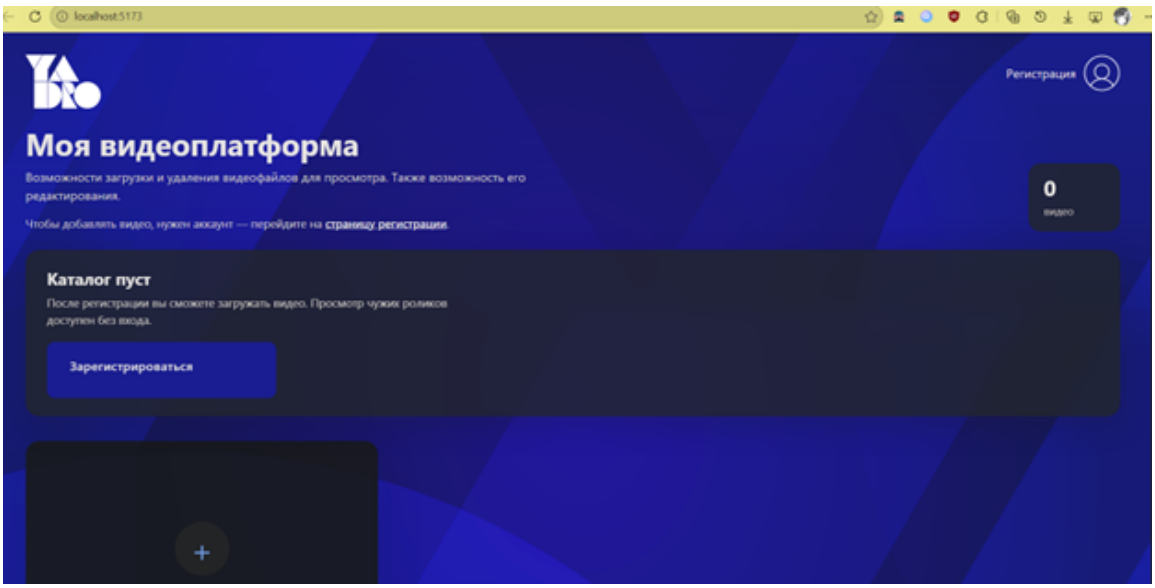

Рисунок 1 — Главная страница — каталог видео для незарегистрированного пользователя

Страница регистрации показана на рисунке 2. Она включает форму ввода данных пользователя и возможность получения кода доступа.

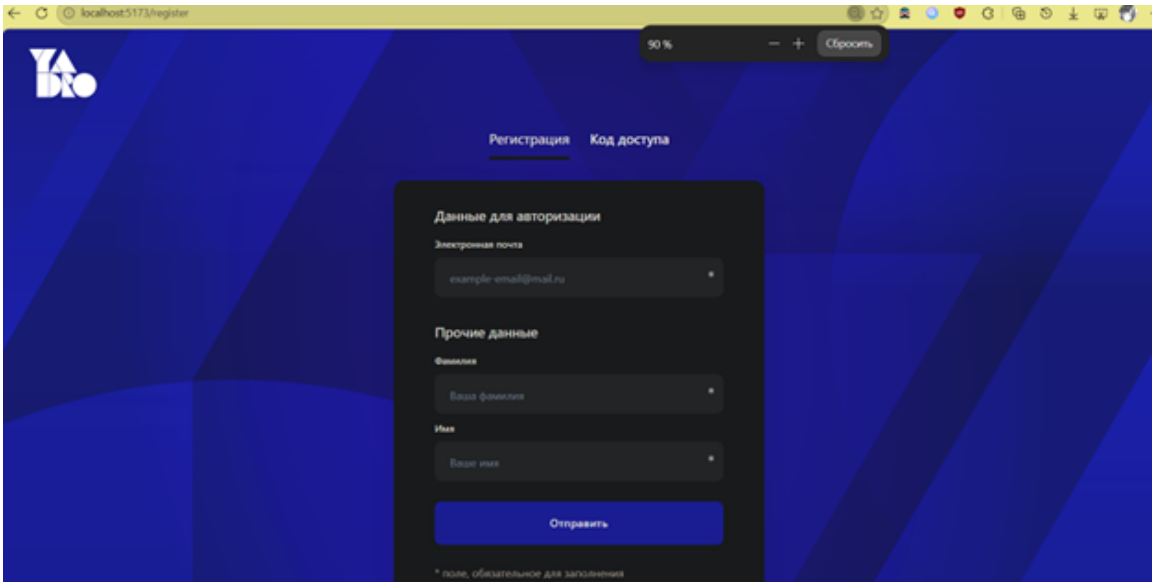

Рисунок 2 — Страница регистрации пользователя

После добавления видео в систему каталог заполняется карточками, содержащими информацию о роликах и действия для работы с ними (рисунок 3).

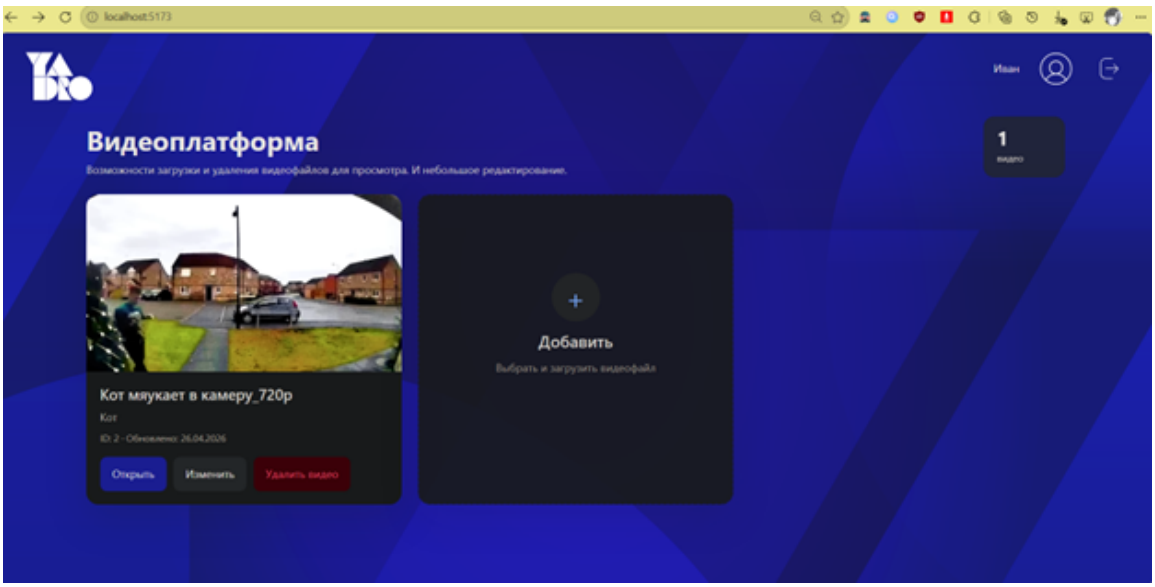

Рисунок 3 — Каталог с загруженными видео

На странице просмотра видео (рисунок 4) отображается сам видеоплеер и блок комментариев, позволяющий взаимодействовать с контентом.

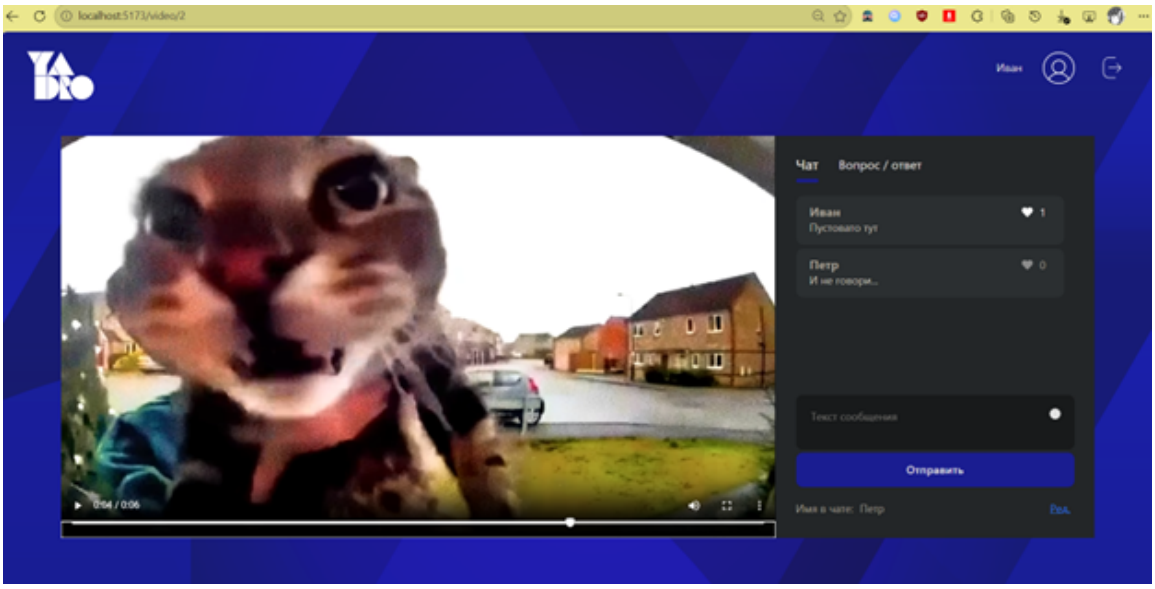

Рисунок 4 — Страница просмотра видео с комментариями

3 ВЫВОДЫ

В ходе выполнения расчётно-графической работы была разработана тестовая видеоплатформа, реализующая взаимодействие клиентской и серверной частей. Пользователь имеет возможность зарегистрироваться, просматривать каталог видео, загружать новые файлы, редактировать информацию о них и переходить к странице просмотра с комментариями.

В процессе работы были применены современные технологии веб-разработки: FastAPI для серверной части, React для клиентской части, а также PostgreSQL для хранения данных. Реализовано взаимодействие между компонентами системы через REST API.

Дополнительно были получены практические навыки настройки окружения, работы с Docker, а также устранения типичных проблем, возникающих при разработке веб-приложений, таких как настройка CORS, согласование адресов между фронтендом и бэкендом и организация работы с файловой системой.

В результате выполненной работы удалось закрепить знания по созданию полноценных веб-приложений и понять основные этапы их разработки — от проектирования структуры до реализации и тестирования.

В качестве дальнейшего развития проекта можно рассмотреть внедрение механизма аутентификации на основе JWT, расширение функционала пользовательского взаимодействия, а также добавление автоматизированного тестирования.